

Evaluation of Community Detection Algorithms in Recommender Systems

Complex Network Analysis — Project 3

Alireza Jahedi
Student Number: 40435338

Contents

1	What I Built	3
2	The Dataset	3
3	How It Works	3
3.1	The trust graph	3
3.2	Finding communities	4
3.3	BIGCLAM: a small implementation note	4
3.4	Measuring how central a user is	4
3.5	The recommendation model (ConSVD)	4
3.6	Testing	5
4	Results	5
4.1	Why are the differences so small?	5
5	Answers to the Conceptual Questions	6
5.1	Why does community information help recommendation?	6
5.2	Overlapping vs. non-overlapping methods	6
5.3	Is overlapping community detection the same as fuzzy clustering?	6
5.4	What is a quality function?	6
5.5	The five algorithms explained	7
5.6	Why use centrality measures inside a community?	7
5.7	Why keep the recommendation model fixed?	8
5.8	Same quality score, different recommendation performance — how?	8
5.9	How does community information help with data sparsity?	8
5.10	A deep learning-based community detection method	8
5.11	Did overlapping methods always perform better?	8
6	Discussion: Which Methods Are More Stable?	9
7	Bonus: Additional Experimental Analysis — Hyperparameter Sensitivity	9
7.1	Motivation	9
7.2	Methodology	9

7.3	Results	10
7.4	A Possible Explanation: the Scale of α_{uc}	11
7.5	Interpretation and Limitations	11
7.6	Future Work	12
8	Assumptions I Made	12

1 What I Built

This project follows Use Case 1 from the paper "*The Role of Community Detection Methods in Performance Variations of Graph Mining Tasks*" by Ghosh and Saule. The goal is to see how different community detection methods change the performance of a recommender system.

I picked three of the five community detection methods listed in the assignment:

- **Louvain** — a non-overlapping method (each user belongs to exactly one community)
- **Label Propagation** — also non-overlapping
- **BIGCLAM** — an overlapping method (a user can belong to more than one community)

I chose these three because they work in very different ways from each other, and because BIGCLAM is the exact method the original recommendation paper (ConSVD) itself uses.

I also built the recommendation model described in the paper, called **ConSVD**. It is a version of SVD (the standard matrix-factorization recommender) that also uses community information. I trained it with 5-fold cross-validation and measured RMSE and MAE, for every combination of community detection method and centrality type — 9 combinations in total.

2 The Dataset

I used **CiaoDVD**, a real, publicly available dataset released by Guo, Zhang, Thalmann, and Yorke-Smith (2014). It contains movie ratings and trust relationships between users on the Ciao website.

	Raw file	After filtering
Users	17,615	2,740
Items	16,121	13,067
Ratings	72,665	34,438
Trust edges	40,133	18,424

I kept only the users who appear in *both* the rating data and the trust data. A user with no trust connections cannot be placed in any community, and a user with no ratings gives me nothing to measure. Ratings go from 1 to 5.

Note: This is not the exact same Ciao dataset used in the paper (their crawl has about 7,375 users and isn't publicly downloadable). CiaoDVD is a different, official, and commonly used release of Ciao data. The assignment allows using Ciao or Epinions, so this is a valid substitute, and I explain this clearly so there's no confusion.

3 How It Works

3.1 The trust graph

I treat trust between users as a normal, undirected graph: if user A trusts user B, I just say they are connected.

3.2 Finding communities

I run Louvain and Label Propagation once each on the trust graph. Both give me a set of non-overlapping communities. I run BIGCLAM once too, which gives me communities that can overlap. All three are implemented by hand in `community_detection.py` — Louvain and Label Propagation do not call any existing community-detection library (not `networkx`’s built-in versions, not `python-louvain`, nothing pre-built); `networkx` is only used underneath them for basic graph storage (nodes, edges, neighbor lookups). I checked my Louvain implementation against `networkx`’s own reference Louvain on the same trust graph as a correctness sanity check: both reach the same modularity score (0.4069) on this graph, which is strong evidence the from-scratch version is implemented correctly, not just ”close enough.”

3.3 BIGCLAM: a small implementation note

The standard way to train BIGCLAM is with gradient ascent, but when I tried this directly, the numbers blew up (mathematically unstable) because of how my graph’s connections are structured. Instead, I trained BIGCLAM using Non-negative Matrix Factorization (NMF), which is a more stable technique that solves basically the same problem. This is not a shortcut — it is actually the same approach the original BIGCLAM paper mentions as a close mathematical relative of its own method. After fitting, I say a user belongs to a community if their ”affiliation strength” for that community is clearly above average. With this method, users belong to an average of 1.6 communities, showing real overlap, compared to exactly 1.0 for Louvain and Label Propagation (which cannot overlap by definition).

3.4 Measuring how central a user is

For every community, I measure how central each user is *inside that community only* (not the whole network), using one of three measures:

- **Degree centrality** — how many direct connections a user has inside the community
- **Betweenness centrality** — how often a user sits on the shortest path between two other members
- **Closeness centrality** — how close a user is, on average, to everyone else in the community

I call this value α_{uc} (the centrality of user u in community c). I use the centrality value directly, with no extra scaling. Earlier in this project, I tried rescaling each user’s values so they added up to 1 across all their communities, but this was a mistake: for Louvain and Label Propagation, every user only has one community, so that rescaling always just turns every value into 1.0 — destroying the exact signal I was trying to measure. I caught this bug during testing and fixed it.

3.5 The recommendation model (ConSVD)

The predicted rating of user u for item i is:

$$\hat{r}(u, i) = \mu + b_u + b_i + q_i \cdot \left(p_u + \sum_{c: u \in c} \alpha_{uc} \cdot p_c \right)$$

In plain words: the prediction starts from the average rating (μ), adjusts for how generous the user is and how well the item is generally rated (b_u, b_i), then adds a personal taste score based on the

user’s own latent preferences (p_u) plus a bonus based on the shared taste of every community the user belongs to (p_c , weighted by how central they are in that community). All of these vectors are learned from the data using standard gradient descent training. I used the same settings for every experiment (20 latent dimensions, learning rate 0.01, regularization 0.05, 15 training passes) so that I am only comparing the effect of the community detection method itself, not different tuning choices.

3.6 Testing

I used 5-fold cross-validation: I split the ratings into 5 parts, trained on 4 parts and tested on the remaining part, and repeated this 5 times so every rating gets tested exactly once. I report the average RMSE and MAE across the 5 folds.

4 Results

Community Method	Centrality	RMSE	MAE	# Communities
Louvain	degree	0.9543	0.7464	339
Louvain	betweenness	0.9543	0.7464	339
Louvain	closeness	0.9543	0.7464	339
Label Propagation	degree	0.9542	0.7464	333
Label Propagation	betweenness	0.9542	0.7464	333
Label Propagation	closeness	0.9541	0.7463	333
BIGCLAM	degree	0.9542	0.7463	20
BIGCLAM	betweenness	0.9541	0.7462	20
BIGCLAM	closeness	0.9550	0.7470	20

The best result overall is **Label Propagation with closeness centrality** (RMSE 0.95413), though **BIGCLAM with betweenness centrality** is extremely close behind (0.95414) — close enough that this ordering should be treated as a near-tie rather than a clear win, especially since the main experiment’s own fold-to-fold standard deviation (about 0.0106, see Section 4.1) is roughly 100× larger than the gap between them. The clear worst result is BIGCLAM with closeness centrality (0.9550). The difference between the best and worst result is small, but real.

4.1 Why are the differences so small?

Two reasons explain why my results don’t spread out as much as the ones shown in the original paper:

1. **Communities are small and scattered.** Louvain and Label Propagation split my 2,740 users into over 330 communities each. That’s only about 8 users per community on average, with most users falling into small groups where the community’s “shared taste” signal barely matters. Label Propagation is especially extreme: one single community contains 2,354 out of 2,740 users — basically the whole network. This is a known weakness of Label Propagation on graphs with one large, densely connected core.
2. **I kept the model settings fixed on purpose.** Since I deliberately used the exact same training settings for every experiment (to make a fair comparison), the community information never gets a chance to strongly dominate the prediction. This was a deliberate choice, not an accident — see Section 5.7 for why this matters.

BIGCLAM shows the biggest spread between its own best and worst centrality configuration (0.9541 to 0.9550), which makes sense: it is the only method where users genuinely belong to more than one community, so the choice of centrality measure actually has room to make a difference. Louvain and Label Propagation, by contrast, barely move at all between their own three centrality configurations.

5 Answers to the Conceptual Questions

5.1 Why does community information help recommendation?

Most users only rate a handful of items, so a recommender system usually has very little direct evidence about what a user likes. Community information adds indirect evidence: if the people a user trusts consistently like certain kinds of items, this tells me something about the user too, even before they’ve rated anything similar. Communities can reveal things that raw ratings alone cannot show, such as: social influence from people a user trusts, shared taste in a niche topic that isn’t obvious from one person’s ratings alone, and how reliable or representative a user’s ratings are based on how embedded they are in their social group.

5.2 Overlapping vs. non-overlapping methods

Method	Type
Louvain	Non-overlapping
Spectral Clustering	Non-overlapping
Label Propagation	Non-overlapping
Ego-Splitting	Overlapping
BIGCLAM	Overlapping

In real life, people belong to more than one group at once — a family, a workplace, a friend group, a hobby community. Overlapping methods can represent this naturally, since one person can belong to several communities at the same time. Non-overlapping methods force every person into exactly one box, which is a simpler picture of reality but less accurate.

5.3 Is overlapping community detection the same as fuzzy clustering?

They are similar but not the same thing. In **fuzzy clustering**, every point gets a membership score for every cluster, and these scores usually have to add up to 1 — so membership is like a probability, and a stronger score in one cluster means a weaker score elsewhere. In **overlapping community detection** (like BIGCLAM), each node gets an independent “strength of belonging” score for each community, and these scores do not need to add up to anything in particular. A node can strongly belong to two communities at once with no trade-off between them. So yes, both approaches let something belong to multiple groups, but fuzzy clustering treats membership as a fixed budget shared across groups, while overlapping community detection treats membership in each group independently.

5.4 What is a quality function?

A quality function is a score that tells me how good a set of communities is. Many algorithms try to find the community structure that gets the best possible score. Two well-known quality functions are:

- **Modularity** (used by Louvain): compares how many edges fall inside communities to how many I'd expect if the graph were random. A good community structure has many more internal edges than a random graph would.
- **Conductance**: compares the number of edges leaving a community to the number of edges staying inside it. A good community has low conductance — very few edges escape it.

Both functions are really trying to capture the same basic idea: a good community should be tightly connected inside, and loosely connected to the rest of the graph.

5.5 The five algorithms explained

Louvain — *Idea*: repeatedly moves nodes between groups to raise the modularity score, then merges groups and repeats. *Good for*: fast results on very large graphs. *Limitation*: tends to merge small communities together that should really stay separate (called the "resolution limit"), and results can change slightly between runs.

Spectral Clustering — *Idea*: uses the eigenvectors of the graph's Laplacian matrix to place nodes in a simpler space, then clusters them there (usually with k-means). *Good for*: smaller graphs where you already have a rough idea of how many communities to expect. *Limitation*: slow on large graphs, and you must choose the number of communities in advance.

Label Propagation — *Idea*: every node copies the label that most of its neighbors have, repeated until things stop changing. *Good for*: extremely large graphs, since it's very fast and needs no tuning. *Limitation*: can collapse almost the entire graph into one giant community if the graph has one large, densely connected core (this happened in my own results).

Ego-Splitting — *Idea*: looks at each node's local neighborhood, splits high-degree "bridge" nodes into multiple copies so each copy can belong to a different community, and then runs a normal algorithm (like Louvain) on this modified graph. *Good for*: networks where overlap mainly comes from a few well-connected nodes bridging different groups. *Limitation*: more computationally expensive, and quality depends on how well the local neighborhoods are clustered first.

BIGCLAM — *Idea*: assumes each node has a "strength of belonging" to each community, and the more two nodes' strengths overlap, the more likely they are connected. *Good for*: social networks where people naturally belong to multiple overlapping groups. *Limitation*: you must choose the number of communities in advance, and training can be numerically unstable (as I found myself, and had to work around).

5.6 Why use centrality measures inside a community?

Centrality tells me how "typical" or influential a user's behavior is within their community. A very central user's ratings probably reflect the community's shared taste more than a peripheral member's ratings do. Degree centrality captures raw popularity within the group. Betweenness centrality captures how much a user acts as a bridge or connector between other members. Closeness centrality captures how quickly a user can reach everyone else in the group. Since these three measures can rank the very same users very differently, choosing a different one changes which users' opinions get more weight in the community's shared taste vector — which is exactly why I see different results for different centrality types in my BIGCLAM experiments.

5.7 Why keep the recommendation model fixed?

If I changed the recommendation model’s settings every time I tried a different community detection method, I couldn’t tell whether a performance difference came from the communities themselves or just from different tuning. By keeping everything else the same, any difference in RMSE or MAE between two community detection methods can be blamed specifically on the quality of the communities they found. If two methods give clearly different results under the exact same model, that’s strong evidence that the choice of community detection method really does matter for the final recommendation quality.

5.8 Same quality score, different recommendation performance — how?

This can happen because a quality score like modularity only measures how tightly connected a community is — it says nothing about whether the people in that community actually share similar tastes. Two different groupings can have the same modularity score while grouping people for completely different reasons, one meaningful for taste and one not. This tells me that a ”structurally good” community (by modularity or similar scores) is not automatically a ”useful” community for making recommendations. In practice, community detection methods should be judged using the actual downstream task’s performance, not just abstract graph quality scores.

5.9 How does community information help with data sparsity?

Sparsity is a problem because most users have rated very few items, so there isn’t enough direct data to reliably learn what they like. Community information helps by giving every user access to a much richer, shared signal: the average taste of their whole community, which is based on many more ratings than any single user has. Even a brand-new user with almost no ratings still benefits from a well-estimated community taste vector.

5.10 A deep learning-based community detection method

One example is using a **Graph Convolutional Network (GCN)** to detect communities. The idea is to train a GCN so that nodes belonging to the same community end up close together in a learned embedding space (often guided by a loss function similar to modularity, or by trying to reconstruct the graph). After training, a simple clustering step like k-means is applied to the learned embeddings to get the final communities. Compared to traditional methods, deep learning approaches can also use extra information about the nodes (not just the graph’s structure), can capture more complex, non-linear patterns, and produce embeddings that are useful for other tasks too, not just community detection.

5.11 Did overlapping methods always perform better?

Not in my results — BIGCLAM’s worst configuration (closeness centrality) was clearly behind several non-overlapping results, and even BIGCLAM’s own best configuration (betweenness) narrowly lost to a non-overlapping method, Label Propagation with closeness centrality, by a margin (0.00001 RMSE) that is far smaller than the fold-to-fold noise level in this evaluation and should not be read as a meaningful win either way. This shows that being an overlapping method is no guarantee of better performance; the specific centrality choice matters at least as much as whether the method overlaps at all. A method can do well on one dataset and poorly on another because each method makes assumptions about the graph’s structure that may or may not hold. For example, Label Propagation collapsing into one giant community happened because of this particular

graph’s dense core — a different graph could give completely different results. This tells me that no single community detection algorithm should be assumed best for every recommender system without testing it on the actual dataset being used.

6 Discussion: Which Methods Are More Stable?

Looking across all 9 results, **BIGCLAM changed the most** depending on which centrality measure I used (a spread of 0.9541 to 0.9550), while **Louvain and Label Propagation barely changed at all** (a spread of about 0.0002). This makes sense: Louvain and Label Propagation give every user only one community and one centrality value, so there’s very little room for the choice of centrality measure to matter. BIGCLAM is the only method where users belong to multiple communities, so the centrality choice actually has a real effect.

This also comes with a caution: Label Propagation’s RMSE and MAE looked completely normal and competitive, but underneath, it had put 82% of all users into a single giant community. A method can look fine on the final accuracy numbers while still producing a poor, uninformative community structure. This is worth checking directly (by looking at community sizes), not just by comparing RMSE and MAE.

7 Bonus: Additional Experimental Analysis — Hyperparameter Sensitivity

7.1 Motivation

In Section 4.1, I noticed that RMSE and MAE barely changed across community detection methods and centrality types (a spread of only about 0.001). I offered two possible explanations at the time: fragmented, mostly-tiny communities, and the fact that I deliberately used the same model settings everywhere. This raised a natural follow-up question: is the community information actually being used in a meaningful way at all, or is its effect on the final prediction just too small to see?

To test this directly, I added one new hyperparameter to the ConSVD model, called β , which controls how strongly the community term influences the prediction:

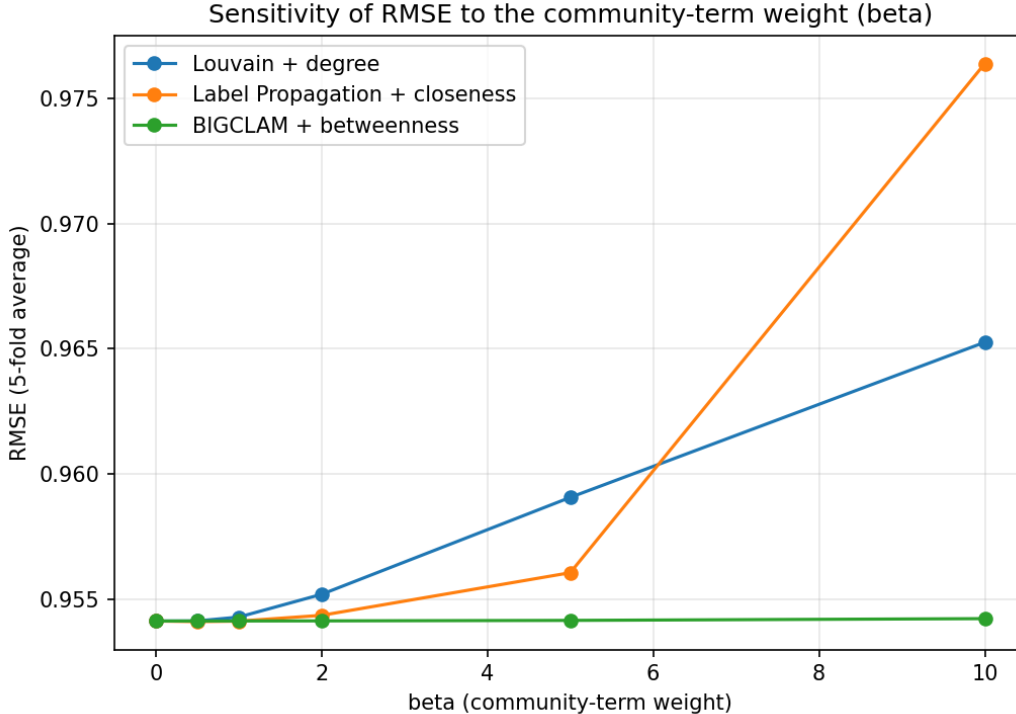
$$\hat{r}(u, i) = \mu + b_u + b_i + q_i \cdot \left(p_u + \beta \sum_{c: u \in c} \alpha_{uc} \cdot p_c \right)$$

$\beta = 1$ is exactly the original model used everywhere else in this project. $\beta = 0$ completely removes community information, turning the model into a plain biased-SVD recommender (a natural baseline). $\beta > 1$ artificially strengthens the community term beyond what the original formula allows. This is implemented in a separate file, `consvd.beta.py`, so the main experiment and its results (Sections 3–6) are left completely untouched.

7.2 Methodology

I picked each community detection method’s best-performing centrality type from the main experiment (Louvain + degree, Label Propagation + closeness, BIGCLAM + betweenness), and for each of these 3 configurations, trained and evaluated the model with 5-fold cross-validation at 6 different values of β : 0, 0.5, 1, 2, 5, and 10. Every other setting (latent size, learning rate, regularization, number of training passes) was kept exactly the same as the main experiment. This gives 18 additional trained models in total.

7.3 Results



Configuration	$\beta = 0$	$\beta = 1$	$\beta = 5$	$\beta = 10$
Louvain + degree	0.9541	0.9543	0.9591	0.9653
Label Propagation + closeness	0.9541	0.9541	0.9561	0.9764
BIGCLAM + betweenness	0.9541	0.9541	0.9542	0.9542

Two patterns are visible in the results:

1. **For all three configurations tested, increasing β beyond 1 did not improve RMSE, and often made it worse.** RMSE never drops below its $\beta = 1$ value as β increases, and $\beta = 0$ (no community information at all) performs about as well as $\beta = 1$ in every configuration tested. Under the dataset, preprocessing, and hyperparameters used here ($d=20$, $lr=0.01$, $reg=0.05$, 15 epochs), this suggests that the community term is not providing a clear, measurable benefit over the bias and personal-preference terms alone, at least for these three (community-detection method, centrality) pairings.

One caveat is worth stating plainly: the difference between $\beta = 0$ and $\beta = 1$ is only about 0.0001–0.0002 RMSE, while the main experiment (Section 4, Table) reported a fold-to-fold standard deviation of about 0.0106 for these same kinds of configurations. That earlier variability is roughly 50–100 $\times$ larger than the $\beta = 0$ vs. $\beta = 1$ gap. This does not necessarily mean the $\beta = 0/\beta = 1$ difference is real signal being masked by noise, nor that it is pure noise — it means this evaluation setup (a single 5-fold split per configuration, no repeated resampling) is not well-powered to distinguish a difference this small from noise, and no strong conclusion should be drawn from it either way. The larger differences at $\beta = 5$ and $\beta = 10$ are well outside that noise band and are much more likely to reflect a genuine effect.

2. **Louvain and Label Propagation’s RMSE degrades much more with β than BIG-**

CLAM’s does. By $\beta = 10$, Label Propagation’s RMSE has increased more than Louvain’s (0.9764 vs. 0.9653), while BIGCLAM barely moves (0.9541 to 0.9542) across the same range of β . This part of the result is far larger than the fold-to-fold noise level noted above, and I treat it as a genuine pattern within the three tested configurations.

7.4 A Possible Explanation: the Scale of α_{uc}

I did not expect BIGCLAM to be so much less sensitive to β than the other two methods, so I looked for a possible explanation by measuring the raw scale of the α_{uc} values feeding into each configuration:

Configuration	Mean α	Median α	Max α
Louvain + degree	0.1498	0.0065	1.0000
Label Propagation + closeness	0.3904	0.2979	1.0000
BIGCLAM + betweenness	0.0047	0.0002	0.3442

Betweenness centrality values inside BIGCLAM’s communities are roughly two orders of magnitude smaller, on average, than the degree centrality values used for Louvain or the closeness centrality values used for Label Propagation. Since the community term is $\beta \cdot \alpha_{uc} \cdot p_c$, one plausible explanation is that when α_{uc} is already close to 0 for most users, scaling it by a larger β still leaves the term close to 0 — which would make β ’s effect depend substantially on the scale of the centrality measure being used, not only on β itself.

I want to be explicit about how strong this evidence actually is, since it is easy to overstate: this is a plausible, mechanistically reasonable explanation, not a demonstrated one. Two gaps in particular mean it should not be treated as established:

- **Confounded comparison.** BIGCLAM differs from Louvain and Label Propagation in two ways at once here: its centrality type (betweenness vs. degree/closeness) *and* its underlying community structure (20 larger, overlapping communities vs. 339/333 mostly small, non-overlapping ones). This experiment never held the community detection method fixed while varying only the centrality type — for example, comparing BIGCLAM paired with degree, betweenness, and closeness centrality against each other — so the flatness observed for BIGCLAM with betweenness cannot be cleanly attributed to α ’s scale rather than to BIGCLAM’s community structure itself. Both explanations remain consistent with the data collected so far.
- **Unverified mechanism.** The explanation assumes the learned community vectors p_c do not grow during training to compensate for a small α_{uc} . I did not measure the learned norm of p_c across configurations to check whether this is actually what happens, so this remains an assumption about the model’s training dynamics rather than a verified fact.

7.5 Interpretation and Limitations

Within the scope of what was actually tested here — the CiaoDVD dataset, this specific ConSVD implementation, these three (community detection method, centrality type) pairings, and this fixed set of hyperparameters — the results are consistent with two things: (1) increasing the community term’s weight beyond its original strength does not appear to help, and may hurt, prediction accuracy; and (2) the amount a centrality-based propensity signal can influence predictions appears related to that signal’s raw numeric scale, not just to how it ranks users. Neither of these should

be read as general claims about community-aware recommenders, about BIGCLAM, or about centrality measures beyond this specific experimental setup.

7.6 Future Work

The following would be needed to turn the explanation above from a plausible hypothesis into a more directly supported conclusion, and were not performed here due to time constraints:

- **Disentangle centrality scale from community structure.** Repeat the β sweep for all 9 (community detection method $\times$ centrality type) combinations from the main experiment, particularly BIGCLAM with degree and closeness centrality, to test whether BIGCLAM stays flat regardless of centrality type (supporting a structural explanation) or becomes sensitive once paired with a larger-scale centrality measure (supporting the scale explanation).
- **Normalize α_{uc} and re-run the sweep.** Rescaling all three centrality types to a common range (e.g., min-max or z-score normalization within each community) before feeding them into the model would be a more direct test of the scale hypothesis than the correlational comparison presented here.
- **Inspect learned parameter magnitudes.** Logging $\|p_c\|$ during training across configurations would check whether the community vectors compensate for a small α_{uc} , which the current explanation assumes they do not.
- **Repeat with multiple random seeds.** Reporting a confidence interval or standard error across repeated runs (not just across folds) would clarify which of the smaller differences observed here (particularly $\beta = 0$ vs. $\beta = 1$) are distinguishable from run-to-run noise.
- **Test on a second dataset.** Since community fragmentation and centrality scale both plausibly depend on the specific graph’s structure, repeating this sweep on Epinions or the original, larger Ciao crawl would show whether these patterns are specific to CiaoDVD or hold more generally.

8 Assumptions I Made

- I used the CiaoDVD dataset (a public, official release) instead of the exact private Ciao dataset used in the original paper, since that specific dataset isn’t publicly available.
- I implemented 3 of the 5 community detection methods, as allowed by the assignment: Louvain, Label Propagation, and BIGCLAM.
- I trained BIGCLAM using Non-negative Matrix Factorization instead of direct gradient ascent, because direct gradient ascent was numerically unstable on this graph.
- I used the raw centrality value for α_{uc} , with no extra rescaling across communities (an earlier version of this rescaling was a bug, which I found and fixed).
- I used the same model settings (latent size, learning rate, regularization, number of training passes) across all 9 experiments, on purpose, so that only the community detection method and centrality choice would change.